

PROXION '26

Query Quest — SQL Solutions

Team: Binary Knights

Abdul Hayy Khan
Muhammad Fasih
Muhammad Amaan
Muhammad Nihal

SENTEC

08–10 May 2026

Q1. Second Highest Distinct Salary

Tables: *Employees*

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);
```

Q2. Users with 3+ Consecutive Login Days

Tables: *Logins*

```
SELECT DISTINCT l1.user_id
FROM Logins l1
JOIN Logins l2 ON l1.user_id = l2.user_id
    AND l2.login_date = DATE_ADD(l1.login_date, INTERVAL 1 DAY)
JOIN Logins l3 ON l1.user_id = l3.user_id
    AND l3.login_date = DATE_ADD(l1.login_date, INTERVAL 2 DAY);
```

Q3. Duplicate Employee Names

Tables: *Employees*

```
SELECT employee_name, COUNT(*) AS occurrences
FROM Employees
GROUP BY employee_name
HAVING COUNT(*) > 1;
```

Q4. Managers Supervising ≥ 5 Employees

Tables: *Employees*

```
SELECT manager_id
FROM Employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
HAVING COUNT(employee_id) >= 5;
```

Q5. Highest Earner Per Department

Tables: *Employees, Departments*

```
SELECT d.department_name, e.employee_name, e.salary
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM Employees e2
    WHERE e2.department_id = e.department_id
);
```

Q6. Customers Who Never Placed an Order

Tables: *Customers, Orders*

```
SELECT c.customer_name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q7. Dates with Higher Temperature than Previous Day

Tables: *Weather*

```
SELECT w1.recordDate
FROM Weather w1
JOIN Weather w2
    ON w2.recordDate = DATE_SUB(w1.recordDate, INTERVAL 1 DAY)
WHERE w1.temperature > w2.temperature;
```

Q8. Products with Above-Average Total Sales Quantity

Tables: *Products, Sales*

```
SELECT p.product_name
FROM Products p
JOIN Sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name
HAVING SUM(s.quantity) > (
    SELECT AVG(total_qty)
    FROM (
        SELECT SUM(quantity) AS total_qty
        FROM Sales
        GROUP BY product_id
    ) AS sub
);
```

Q9. Top Spending Customer Per Month

Tables: *Customers, Orders*

```
SELECT MONTHNAME(o.order_date) AS month,
       c.customer_name,
       SUM(o.amount) AS total_spent
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY YEAR(o.order_date), MONTH(o.order_date), o.customer_id
HAVING SUM(o.amount) = (
    SELECT MAX(monthly_total)
    FROM (
        SELECT customer_id,
               YEAR(order_date) AS yr,
               MONTH(order_date) AS mo,
               SUM(amount) AS monthly_total
        FROM Orders
        GROUP BY customer_id, YEAR(order_date), MONTH(order_date)
    ) AS sub
    WHERE sub.yr = YEAR(o.order_date)
    AND sub.mo = MONTH(o.order_date)
);
```

Q10. Employees Earning More Than Their Manager

Tables: *Employees*

```
SELECT e.employee_name
FROM Employees e
JOIN Employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

Q11. Numbers Appearing 3+ Times Consecutively

Tables: *Logs*

```
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM Logs l1
JOIN Logs l2 ON l2.id = l1.id + 1 AND l2.num = l1.num
JOIN Logs l3 ON l3.id = l1.id + 2 AND l3.num = l1.num;
```

Q12. Running Total of Sales by Date

Tables: *Sales*

```
SELECT sale_date,
       SUM(quantity) OVER (ORDER BY sale_date
                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                           ) AS running_total
FROM Sales
ORDER BY sale_date;
```

Q13. Department-wise Salary Ranking (DENSE_RANK)

Tables: *Employees, Departments*

```
SELECT e.employee_name,  
       e.salary,  
       DENSE_RANK() OVER (PARTITION BY e.department_id  
                           ORDER BY e.salary DESC) AS rank_pos  
FROM Employees e  
JOIN Departments d ON e.department_id = d.department_id  
ORDER BY e.department_id, rank_pos;
```

Q14. Median Salary

Tables: *Employees*

```
SELECT AVG(salary) AS median_salary  
FROM (  
    SELECT salary,  
           ROW_NUMBER() OVER (ORDER BY salary) AS rn,  
           COUNT(*) OVER () AS total  
    FROM Employees  
    ) AS ranked  
WHERE rn IN (FLOOR((total + 1) / 2), CEIL((total + 1) / 2));
```

Q15. Salary Gap (Max - Min) Per Department

Tables: *Employees, Departments*

```
SELECT d.department_name,  
       MAX(e.salary) - MIN(e.salary) AS salary_gap  
FROM Employees e  
JOIN Departments d ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name;
```

Q16. Customers with Orders in 3 Consecutive Months

Tables: *Customers, Orders*

```
SELECT DISTINCT c.customer_name  
FROM Customers c  
JOIN Orders o1 ON c.customer_id = o1.customer_id  
JOIN Orders o2 ON c.customer_id = o2.customer_id  
    AND PERIOD_DIFF(  
        DATE_FORMAT(o2.order_date, '%Y%m'),  
        DATE_FORMAT(o1.order_date, '%Y%m')  
    ) = 1  
JOIN Orders o3 ON c.customer_id = o3.customer_id  
    AND PERIOD_DIFF(  
        DATE_FORMAT(o3.order_date, '%Y%m'),  
        DATE_FORMAT(o1.order_date, '%Y%m')  
    ) = 2;
```

Q17. Top 3 Distinct Salaries Per Department ★

Tables: *Employees, Departments*

```
SELECT d.department_name, e.salary
FROM Departments d
JOIN (
    SELECT department_id,
           salary,
           DENSE_RANK() OVER (PARTITION BY department_id
                               ORDER BY salary DESC) AS dr
    FROM Employees
) e ON d.department_id = e.department_id
WHERE e.dr <= 3
ORDER BY d.department_name, e.salary DESC;
```

Q18. Product Pairs Ordered Together > 10 Times

Tables: *Orders, OrderDetails, Products*

```
SELECT p1.product_name AS product1,
       p2.product_name AS product2
FROM OrderDetails od1
JOIN OrderDetails od2
    ON od1.order_id = od2.order_id
   AND od1.product_id < od2.product_id
JOIN Products p1 ON od1.product_id = p1.product_id
JOIN Products p2 ON od2.product_id = p2.product_id
GROUP BY od1.product_id, od2.product_id,
         p1.product_name, p2.product_name
HAVING COUNT(*) > 10;
```

Q19. Longest Login Streak Per User

Tables: *Logins*

```
SELECT user_id, MAX(streak) AS longest_streak
FROM (
    SELECT user_id,
           COUNT(*) AS streak
    FROM (
        SELECT user_id,
               login_date,
               DATE_SUB(login_date,
                        INTERVAL ROW_NUMBER() OVER (
                            PARTITION BY user_id ORDER BY login_date
                        ) DAY
               ) AS grp
        FROM (SELECT DISTINCT user_id, login_date FROM Logins) AS d
    ) AS grouped
    GROUP BY user_id, grp
) AS streaks
GROUP BY user_id;
```

Q20. Employees Sharing Same Salary in Same Department

Tables: *Employees*

```
SELECT e.employee_name, e.salary
FROM Employees e
WHERE (e.department_id, e.salary) IN (
    SELECT department_id, salary
    FROM Employees
    GROUP BY department_id, salary
    HAVING COUNT(*) > 1
)
ORDER BY e.department_id, e.salary;
```

Q21. Month-over-Month Sales Growth %

Tables: *Orders*

```
SELECT MONTHNAME(curr.order_month) AS month,
       ROUND(
           (curr.total - prev.total) / prev.total * 100,
           2
       ) AS growth_percentage
FROM (
    SELECT DATE_FORMAT(order_date, '%Y-%m-01') AS order_month,
           SUM(amount) AS total
    FROM Orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m-01')
) AS curr
JOIN (
    SELECT DATE_FORMAT(order_date, '%Y-%m-01') AS order_month,
           SUM(amount) AS total
    FROM Orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m-01')
) AS prev
ON curr.order_month = DATE_ADD(prev.order_month, INTERVAL 1 MONTH)
ORDER BY curr.order_month;
```

Q22. Missing Customer IDs in Sequential DB

Tables: *Customers*

```
SELECT seq.n AS missing_id
FROM (
    SELECT a.customer_id + b.n AS n
    FROM Customers a
    JOIN (
        SELECT 0 AS n UNION SELECT 1 UNION SELECT 2
        UNION SELECT 3 UNION SELECT 4 UNION SELECT 5
        UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9
    ) b
    WHERE a.customer_id + b.n <= (SELECT MAX(customer_id) FROM Customers)
) seq
LEFT JOIN Customers c ON seq.n = c.customer_id
WHERE c.customer_id IS NULL
AND seq.n >= (SELECT MIN(customer_id) FROM Customers)
ORDER BY seq.n;
```

Q23. Highest Spending Customer Per Product Category

Tables: *Customers, Orders, Products, OrderDetails*

```
SELECT p.category,
       c.customer_name
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.category, c.customer_id, c.customer_name
HAVING SUM(od.quantity * p.price) = (
    SELECT MAX(cat_total)
    FROM (
        SELECT c2.customer_id,
               p2.category,
               SUM(od2.quantity * p2.price) AS cat_total
        FROM Customers c2
        JOIN Orders o2 ON c2.customer_id = o2.customer_id
        JOIN OrderDetails od2 ON o2.order_id = od2.order_id
        JOIN Products p2 ON od2.product_id = p2.product_id
        GROUP BY c2.customer_id, p2.category
    ) AS sub
    WHERE sub.category = p.category
)
ORDER BY p.category;
```

Q24. Dept with Highest Avg Salary (≥ 3 Employees)

Tables: *Employees, Departments*

```
SELECT d.department_name,
       ROUND(AVG(e.salary), 2) AS average_salary
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) >= 3
ORDER BY average_salary DESC
LIMIT 1;
```

Q25. Customers with > 5 Orders in Any 10-Minute Window

Tables: *Orders*

```
SELECT DISTINCT o1.customer_id
FROM Orders o1
JOIN Orders o2
    ON o1.customer_id = o2.customer_id
    AND o2.order_date >= o1.order_date
    AND o2.order_date < DATE_ADD(o1.order_date, INTERVAL 10 MINUTE)
GROUP BY o1.customer_id, o1.order_date
HAVING COUNT(DISTINCT o2.order_id) > 5;
```